

3주차 과제2

👤 사람 ■ 백소정(인공지능대학 사이버보안학과)

패키지 구조

인터페이스 함수 (analyzer.py)

출력 스키마 (schema.py)

체크할 포인트

패키지 구조

```
static_analyzer/  
├─ __init__.py  
├─ analyzer.py          # analyze_static() 엔트리포인트 (오케스트레이터가 이것만 import)  
├─ apk_extractor.py      # apktool/jadx 호출해서 리소스+소스 추출  
├─ manifest_parser.py   # AndroidManifest.xml 파싱 (권한, 컴포넌트)  
├─ cert_analyzer.py     # 서명 인증서 검증/추출  
├─ code_scanner.py      # 디컴파일 코드에서 API 호출·난독화·리플렉션 탐지  
├─ string_extractor.py  # strings 추출 + URL/IP 정규식 매칭  
├─ sdk_detector.py      # 서드파티 SDK 시그니처 매칭  
├─ risk_scorer.py       # 종합 위험도 스코어링  
├─ schema.py            # 출력 dict 타입 정의 (팀 공유용 핵심 파일)  
└─ exceptions.py       # StaticAnalysisError 등 커스텀 예외
```

인터페이스 함수 (analyzer.py)

```
# analyzer.py  
from static_analyzer.schema import StaticAnalysisResult  
  
def analyze_static(apk_path: str) -> StaticAnalysisResult:  
    """  
    APK 정적 분석 수행.  
  
    Args:  
        apk_path: 분석 대상 .apk 파일 경로  
  
    Returns:  
        StaticAnalysisResult (TypedDict, JSON 직렬화 가능)  
  
    Raises:  
        FileNotFoundError: apk_path가 존재하지 않을 때  
        StaticAnalysisError: apktool/jadx 실행 실패 등 치명적 에러  
        (부분 실패는 예외 대신 result["errors"]에 누적하고 계속 진행)  
    """  
    ...
```

아래는 analyzer.py 템플릿 코드

```
# 예시 analyzer.py 코드  
from static_analyzer.apk_extractor import extract_apk  
from static_analyzer.manifest_parser import parse_manifest  
from static_analyzer.cert_analyzer import analyze_cert
```

```

from static_analyzer.code_scanner import scan_code
from static_analyzer.string_extractor import extract_strings
from static_analyzer.sdk_detector import detect_sdks
from static_analyzer.risk_scorer import calculate_risk

def analyze_static(apk_path: str) -> dict:
    errors = []

    # 1. 각 하위 모듈이 "자기 몫"의 원본 데이터를 만들어냄
    extracted = extract_apk(apk_path) # apktool/jadx 실행 결과
    manifest_data = parse_manifest(extracted) # 매니페스트 파싱 결과
    cert_data = analyze_cert(extracted) # 인증서 파싱 결과
    code_data = scan_code(extracted) # 코드 스캔 결과
    strings_data = extract_strings(extracted) # 문자열 추출 결과
    sdks = detect_sdks(extracted)

    # 2. 흩어진 결과들을 risk_scorer가 종합
    risk = calculate_risk(manifest_data, code_data, strings_data)

    # 3. schema.py가 정한 "모양"대로 하나의 dict에 담아서 리턴
    return {
        "meta": extracted["meta"],
        "manifest": manifest_data,
        "certificate": cert_data,
        "code_analysis": code_data,
        "strings": strings_data,
        "third_party_sdks": sdks,
        "risk_score": risk,
        "errors": errors,
    }

```

출력 스키마 (schema.py)

```

from typing import TypedDict, List, Dict

class FileHash(TypedDict):
    md5: str
    sha1: str
    sha256: str

class Meta(TypedDict):
    package_name: str
    version_name: str
    version_code: int
    min_sdk: int
    target_sdk: int
    file_hash: FileHash
    file_size: int

class ManifestInfo(TypedDict):
    permissions: List[str]
    dangerous_permissions: List[str]
    activities: List[str]

```

```

services: List[str]
receivers: List[str]
providers: List[str]
exported_components: List[str]

class CertInfo(TypedDict):
    issuer: str
    subject: str
    valid_from: str
    valid_to: str
    is_self_signed: bool

class CodeAnalysis(TypedDict):
    suspicious_api_calls: List[Dict[str, str]] # {api, location, risk}
    obfuscation_detected: bool
    native_libraries: List[str]
    reflection_usage: bool
    dynamic_code_loading: bool

class StringsInfo(TypedDict):
    urls: List[str]
    ip_addresses: List[str]
    suspicious_strings: List[str]

class StaticAnalysisResult(TypedDict):
    meta: Meta
    manifest: ManifestInfo
    certificate: CertInfo
    code_analysis: CodeAnalysis
    strings: StringsInfo
    third_party_sdks: List[str]
    risk_score: float # 0.0 ~ 1.0
    errors: List[str] # 부분 실패 시 누적, 빈 리스트면 성공

```

- `analyzer.py` → 인터페이스 함수 그 자체가 있는 파일
- `schema.py` → 그 함수의 출력 형식을 정의한 파일

체크할 포인트

1. **errors 필드 넣기:** 정적 분석은 난독화 심한 APK에서 일부 파싱 실패하는 경우가 흔한데, 예외로 전체 죽이지 말고 부분 결과 + 에러 로그로 반환하는 게 오케스트레이터 입장에서 다루기 편함
2. **risk_score는 이 모듈에서 낼지 오케스트레이터가 종합할지:** 동적분석 결과랑 합산해서 최종 스코어 낼 거면 여기서 정적 분석 기준 점수만 내고 최종 종합은 오케스트레이터 뒤편으로 분리하는 게 나을 수도 있음
3. **apk_path 가 실제 파일인지 스트림/바이트인지:** 팀 전체 인터페이스가 파일 경로 기준인지 먼저 통일해야 함

▼ 3번 설명

`analyze_static(apk_path)` 에서 `apk_path` 라는 매개변수에 실제로 뭘 넘길 건지

옵션 1 — 파일 경로 (문자열)

```
result= analyze_static("/home/user/samples/target.apk")
```

디스크에 이미 저장된 apk 파일의 경로만 넘김. 함수 내부에서 `open(apk_path, "rb")` 로 직접 읽음.

옵션 2 — 바이트 스트림 (메모리 상의 데이터)

```
withopen("target.apk", "rb") as f:    apk_bytes= f.read() result= analyze_static(apk_bytes) # bytes 객체 자체를 넘김
```

파일을 디스크에 저장 안 하고, 메모리에 읽어들인 바이트 덩어리를 그대로 넘김.

옵션 3 — 파일 객체/스트림

```
withopen("target.apk", "rb") as f:    result= analyze_static(f) # file-like object 넘김
```